

Unit 2: Advanced PHP and File Management

Unit – 2: Advanced PHP and File Management

2.1 File Handling and Directories

2.1.1 Including files using include and require

2.1.2 File operations: fopen(), fread(), fwrite(), fclose()

2.1.3 File upload using \$_FILES and move_uploaded_file()

2.1.4 File download using PHP headers

2.1.5 Directory operations: opendir(), readdir(), mkdir(), rmdir()

2.2 Forms, Filters, and JSON

2.2.1 Designing and handling HTML forms

2.2.2 Server-side validation techniques

2.2.3 PHP filters: filter_var() and constants

2.2.4 Parsing and generating JSON with json_encode() and json_decode()

2.3 Cookies, Sessions, and Emails

2.3.1 Creating and accessing cookies using setcookie() and \$_COOKIE

2.3.2 Session management with session_start() and \$_SESSION

2.3.3 Sending emails using the mail() function

2.3.4 Email formatting: headers, subject, attachments

2.4 OOP and Exception Handling in PHP

2.4.1 Creating classes and objects

2.4.2 Using constructors and property visibility

2.4.3 Inheritance and method overriding

2.4.4 Exception handling: try, catch, finally, throw

2.4.5 Input validation using regular expressions

Unit 2: Advanced PHP and File Management

2.1 File Handling and Directories

2.1.1 Including Files Using include and require

Purpose:

To reuse code (e.g., headers, footers, menus, database connections) across multiple pages by including external files.

Syntax:

```
include 'filename.php';
```

```
require 'filename.php';
```

Difference:

- include: Shows a warning if the file is missing, and continues execution.
- require: Shows a fatal error and stops script execution if the file is missing.

Practical Example:

header.php

```
<h1>Welcome to My Website</h1>
```

main.php

```
<?php
```

```
include 'header.php'; // or use require
```

```
echo "Main Content Here";
```

```
?>
```

Working with Files in PHP

To create, access, and manipulate files on your web server using the PHP file system functions.

Opening a File with PHP fopen() Function

To work with a file you first need to open the file. The PHP fopen() function is used to open a file.

Syntax:

```
fopen(filename, mode)
```

Example:

Unit 2: Advanced PHP and File Management

```
<?php
    $file = "data.txt"; // Check the existence of file
    if(file_exists($file))
    {
        // Attempt to open the file
        $handle = fopen($file, "r");
    }
else
    {
        echo "ERROR: File does not exist.";
    }

?>
```

Note: check whether a file or directory exists or not before trying to access it, with the PHP `file_exists()` function.

Mode

Modes	What it does
R	Open the file for reading only.
r+	Open the file for reading and writing.
W	Open the file for writing only and clears the contents of file. If the file does not exist, PHP will attempt to create it.
w+	Open the file for reading and writing and clears the contents of file. If the file does not exist, PHP will attempt to create it.
a	Append. Opens the file for writing only. Preserves file content by writing to the end of the file. If the file does not exist, PHP will attempt to create it.
a+	Read/Append. Opens the file for reading and writing. Preserves file content by writing to the end of the file. If the file does not exist, PHP will attempt to create it.
x	Open the file for writing only. Return FALSE and generates an error if the file already exists. If the file does not exist, PHP will attempt to create it.

Unit 2: Advanced PHP and File Management

Modes	What it does
x+	Open the file for reading and writing; otherwise it has the same behavior as 'x'.

fclose() Function:

Closing a File

```
<?php
```

```
$file = "data.txt"; // Check the existence of file

if(file_exists($file))
{
    // Open the file for reading
    $handle = fopen($file, "r") or die("ERROR: Cannot open the file.");

    /* Some code to be executed */

    // Closing the file handle
    fclose($handle);
}
else
{
    echo "ERROR: File does not exist.";
}
```

```
?>
```

fread():

The fread() function can be used to read a specified number of characters from a file.

Syntax:

```
fread(file handle, length in bytes)
```

Unit 2: Advanced PHP and File Management

Example

```
<?php
    $file = "data.txt";
    if(file_exists($file))
    {
        $handle = fopen($file, "r") or die("ERROR: Cannot open the file.");
        $content = fread($handle, "20");
        fclose($handle);
        echo $content;
    }
    else
    {
        echo "ERROR: File does not exist.";
    }
?>
```

Note:

1. we can use filesize(\$filename) at the place of 20 for size of the file in bytes for reading entire file.
2. readfile() function, allows you to read the contents of a file without needing to open it.

fgets():

The fgets() function is used to read a single line from a file.

```
<?php

    $myfile = fopen("webdictionary.txt", "r") or die("Unable to open file!");
    echo fgets($myfile);
    fclose($myfile);

?>
```

feof():

feof() function checks if the "end-of-file" (EOF) has been reached.

```
<?php
$myfile = fopen("webdictionary.txt", "r") or die("Unable to open file!");
// Output one line until end-of-file
while(!feof($myfile)) {
    echo fgets($myfile) . "<br>";
}
fclose($myfile);

?>
```

Unit 2: Advanced PHP and File Management

fwrite():

The first parameter of fwrite() contains the name of the file to write to and the second parameter is the string to be written.

```
<?php
$myfile = fopen("newfile.txt", "w") or die("Unable to open file!");
$txt = "John Doe\n";
fwrite($myfile, $txt);
$txt = "Jane Doe\n";
fwrite($myfile, $txt);
fclose($myfile);
?>
```

Rename(): rename a file or directory

rename(Old filename, New file name)

File upload

You can upload any kind of file like images, videos, ZIP files, Microsoft Office documents, PDFs, as well as executables files and a wide range of other file types.

```
<html>

  <body>

    <?php

$target_path = "uploads/";

$target_path = $target_path.basename( $_FILES['fileToUpload']['name']);

if(move_uploaded_file($_FILES['fileToUpload']['tmp_name'], $target_path)) {

    echo "File uploaded successfully!";

} else{

    echo "Sorry, file not uploaded, please try again!";

}

?>

    <form method="post" enctype="multipart/form-data">
```

Unit 2: Advanced PHP and File Management

Select File:

```
<input type="file" name="fileToUpload"/>

<input type="submit" value="Upload Image" name="submit"/>

</form>

</body>

</html>
```

Note:

basename()->function which is used to return the base name of a file

\$_FILES['fileToUpload']['name']-> file name

\$_FILES['fileToUpload']['type']-> file type

\$_FILES['fileToUpload']['size']-> file size

\$_FILES['fileToUpload']['tmp_name']->file temporary name

File download

```
<?php

$file = "logo_ldg.png"; //Let say If I put the file name Bang.png

echo "<a href='download.php?nama=".$file.">download</a> ";

?>
```

Download.php

```
<?php

$filename = $_GET["nama"];

$contenttype = "application/force-download";

header("Content-Type: " . $contenttype);

header("Content-Disposition: attachment; filename=\"\" . basename($filename) . "\";");
```

Unit 2: Advanced PHP and File Management

```
readfile("your file uploaded path".$filename);  
  
exit();  
  
?>
```

- Header: The header function sets the headers for an HTTP Response given by the server.
- Redirect page.
It is used to redirect a from one web page to another web page in PHP.

header('Location:give your url here');
- Set Content-Type in header response:
PHP defaults to sending Content-Type:text/html. If we want to change the Content-Type
- Content-Disposition
In a regular HTTP response, the Content-Disposition response header is a header indicating if the content is expected to be displayed INLINE in the browser, that is, as a Web page or as part of a Web page, or as an ATTACHMENT, that is downloaded and saved locally.

2.1.4 File Download Using PHP Headers

How it Works:

- PHP can be used to trigger file downloads by sending appropriate HTTP headers.
- Headers control the browser's behavior to treat the content as a file download.

Important Headers:

- Content-Type: Defines the file type (application/octet-stream for any file)
- Content-Disposition: Suggests how the content should be handled (inline or attachment)
- Content-Length: Specifies file size
- readfile(): Reads the file and sends it to output

download.php

```
<?php
```

```
$file = "uploads/sample.txt";
```


Unit 2: Advanced PHP and File Management

```
if(file_exists($file)){  
    header('Content-Description: File Transfer');  
    header('Content-Type: application/octet-stream');  
    header('Content-Disposition: attachment; filename="'.basename($file)."'");  
    header('Content-Length: ' . filesize($file));  
    readfile($file);  
    exit;  
} else {  
    echo "File not found!";  
}  
?>
```

Call this script like: `download.php?file=sample.txt` if dynamic download required.

2.1.5 Directory Operations: `opendir()`, `readdir()`, `mkdir()`, `rmdir()`

1. `opendir()`

- Opens a directory handle for reading contents.
- Returns a directory handle if successful.

Example:

```
$dir = opendir("uploads");
```

2. `readdir()`

- Reads the next entry (file/folder) from the directory handle.
- Returns file/folder names one by one.

Example:

```
while(($file = readdir($dir)) !== false){
```

Unit 2: Advanced PHP and File Management

```
    echo "File: $file<br>";  
}
```

3. mkdir()

- Creates a new directory.
- Syntax:

```
mkdir("folder_name", permissions, recursive);
```

Example:

```
mkdir("mydata", 0777, true); // creates with full access
```

4. rmdir()

- Deletes an empty directory.
- Fails if directory contains files or is not writable.

Example:

```
rmdir("mydata");
```

Create a Directory:

```
<?php  
  
$dirname = "myfolder";  
  
if(!is_dir($dirname)){  
    mkdir($dirname);  
    echo "Directory created."  
} else {  
    echo "Directory already exists."  
}
```

Unit 2: Advanced PHP and File Management

?>

Read Files from a Directory:

```
<?php
```

```
$dir = "uploads";
```

```
if(is_dir($dir)){
```

```
if($dh = opendir($dir)){
```

```
while(($file = readdir($dh)) !== false){
```

```
if($file != '.' && $file != '..'){
```

```
    echo "File: $file <br>";
```

```
    }
```

```
}
```

```
closedir($dh);
```

```
}
```

```
}
```

?>

Remove a Directory:

```
<?php
```

```
$dir = "myfolder";
```

```
if(is_dir($dir)){
```

```
    rmdir($dir);
```

```
    echo "Directory removed.";
```

```
} else {
```

```
    echo "Directory not found.";
```

```
}
```

?>

Unit 2: Advanced PHP and File Management

Summary Table

Function	Description
include()	Includes a file, continues on error
require()	Includes a file, stops on error
fopen()	Opens a file
fwrite()	Writes data to a file
fread()	Reads data from a file
fclose()	Closes the opened file
\$_FILES	Used to handle uploaded file metadata
move_uploaded_file()	Moves uploaded file to a new location
opendir()	Opens a directory handle
readdir()	Reads filenames in a directory
mkdir()	Creates a directory
rmdir()	Removes a directory

2.2 Forms, Filters, and JSON

2.2.1 Designing and Handling HTML Forms

Forms Creating

The HTML <form> tag is used for creating a form for user input. A form can contain textfields, checkboxes, radio-buttons and more. Forms are used to pass user-data to a specified URL.

Attribute:

action: Specifies where to send the form-data when a form is submitted. Value is URL.

method: Specifies the HTTP method to use when sending form-data. It will be get or post.

Unit 2: Advanced PHP and File Management

name: Specifies the name of a form in text format.

target: Specifies where to display the response that is received after submitting the form. It will be `_blank`, `_self`, `_parent`, `_top`.

Element:

The `<form>` element can contain one or more of the following form elements:

`<input>`
`<textarea>`
`<button>`
`<select>`
`<option>`
`<optgroup>`
`<fieldset>`
`<legend>`
`<label>`

[1] < Input >

The `<input>` tag specifies an input field where the user can enter data. `<input>` elements are used within a `<form>` element to declare input controls that allow users to input data. An input field can vary in many ways, depending on the `type` attribute.

Attribute:

value: Specifies the value of an `<input>` element. It may be a *text*.

name: Specifies the name of an `<input>` element. It may be a *text*.

type: Specifies the type `<input>` element to display . it will be `button`, `checkbox`, `color`, `date`, `datetime`, `email`, `file`, `hidden`, `image`, `month`, `number`, `password`, `radio`, `range`, `reset`, `search`, `submit`, `tel`, `text`, `time`, `url`, `week`.

object description

checkbox: Checkboxes are used when you want the user to select one or more options of a limited number of choices

color: used for input fields that should contain a color.

Date: used for input fields that should contain a date.

Datetime: used for input fields that should contain a date and time.

Unit 2: Advanced PHP and File Management

Email: used for input fields that should contain a email

File: used for input fields that should contain a file.

Hidden: used for input fields that should contain hidden field

Image: used for input fields that should contain image

Month: use for input fields that should contain a month and year.

Number: use for input fields that should contain a numeric value

Password: use for input fields that should contain password.

Radio: use when you want the user to select one of a limited number of choices.

Range: use for input fields that should contain value with in a range.

Reset: used for input fields that should reset the value.

Search: used for search fields (a search field behaves like a regular text field).

Submit: used for input fields that should submit the value.

Text: used when you want the user to type letters, numbers, etc. in a form.

Tel: used for input fields that should contain a telephone number.

Time: used for input fields that should contain a time.

url: used for input fields that should contain a url.

Week: used for input fields that should contain a week and year.

[2] <textarea>

An input that allows a large amount of text to be entered, and allows the height of input box to be a specified unlike the standard input tag.

Attribute:

name - The unique name assigned to the form field.

rows - The number of rows of text, defines the vertical size of the text area.

cols - The horizontal size of the text box, defined as the number of characters

(i.e. columns)

Unit 2: Advanced PHP and File Management

[3] <button>

A form button is similar to other form inputs

attributes:

name - Unique name for the button to be used by the action script.

type - The button type, either submit or reset, determines whether the form is to be submitted or cleared upon pressing it.

value - Text that appears on the button, such as OK or Submit.

size - Determines the length (or width) of the button.

[4] <select>

A drop-down list, also referred to as a combo-box, allowing a selection to be made from a list of items.

Attribute:

name - Selector name

size - The minimum size (width) of the selection list, usually not required as the size of the items will define the list size.

multiple - Allows a user to select multiple items from the list, normally limited to one.

[5] <option>

An option tag is needed for each item in the list, and must appear within the select tags. The text to be shown for the option must appear between the option tags.

Attribute:

value - The value is the data sent to the action script with the option is selected. This is not the text that appears in the list.

selected - Sets the default option that is automatically selected when the form is shown.

Unit 2: Advanced PHP and File Management

[6] <optgroup>

The <optgroup> is used to group related options in a drop-down list. If you have a long list of options, groups of related options are easier to handle for a user.

Attribute:

label: Specifies a label for an option-group. It is a text.

[7] <fieldset>

used to group related elements in a form.

Attribute:

disabled: Specifies that a group of related form elements should be disabled.

form: Specifies one or more forms the fieldset belongs to.

name: Specifies a name for the fieldset.

[8] <legend>

The <legend> tag defines a caption for the <fieldset> element.

Example <fieldset>

```
<legend style="float:right">Personalia:</legend>
<label for="fname">First name:</label>
<input type="text" id="fname" name="fname"><br>

<input type="submit" value="Submit">

</fieldset>
```

[9] <label>

The <label> tag defines a label for an <input> element. The for attribute of the <label> tag should be equal to the id attribute of the related element to bind them together.

Attribute:

for: Specifies which form element a label is bound to

Example:

Unit 2: Advanced PHP and File Management

```
<html>

<body>

<form name="input" action="test.html" method="get">

User Login: <input type="text" name="user" ><br>

Password: <input type="password" name="pass">

<br>

Address:<textarea>default null</textarea><br>

Gender: <input type="radio" name="gen" value="male">

<input type="radio" name="gen" value="female" checked><br>

Language:<input type="checkbox" name="language" value="spanish"> I speak Spanish<br>

<input type="checkbox" name="language" value="french"> I speak French <br>

College

<select name="college">

<option value="v">vvk</option>

<option value="p">pts</option>

<option value="d">drb</option>

</select><br>

<input type="submit" value="Submit">

</form>

</body>

</html>
```

Unit 2: Advanced PHP and File Management

2.2.2 Server-side validation techniques

An HTML form contains various input fields such as text box, checkbox, radio buttons, submit button, and checklist, etc. These input fields need to be validated, which ensures that the user has entered information in all the required fields and also validates that the information provided by the user is valid and correct.

It may be :Empty String, Validate String, Validate Numbers, Validate Email, Validate URL, Input length and so on:

It will manage using Php filters & regular expressions

Php filter:

To validate data using filter extension you need to use the PHP's `filter_var()` function.

Functions and Filters

To filter, use one of the following filter function:

`filter_var()`- Filter a single variable with a specified filter.

`filter_var_array()`- Filter several variables with the same or different filters.

`filter_input()`- Get one input variable and filter it.

`filter_input_array` – Get several input variables and filter them with the same or different filters.

Syntax: `filter_var(variable, filter, options)`

The first parameter is the value to be filtered, the second parameter is the ID of the filter to apply, and the third parameter is the array of options related to filter.

ID	Description
<code>FILTER_VALIDATE_BOOLEAN, FILTER_VALIDATE_BOOL</code>	Returns true for "1", "true", "on" and "yes". Returns false otherwise. If <code>FILTER_NULL_ON_FAILURE</code> is set, false is returned only for "0", "false", "off", "no", and "", and null is returned for all non-boolean values.
<code>FILTER_VALIDATE_DOMAIN</code>	Validates whether the domain name label lengths are valid.

Unit 2: Advanced PHP and File Management

ID	Description
FILTER_VALIDATE_EMAIL	Validates whether the value is a valid e-mail address.
FILTER_VALIDATE_FLOAT	Validates value as float, optionally from the specified range, and converts to float on success.
FILTER_VALIDATE_INT	Validates value as integer, optionally from the specified range, and converts to int on success.
FILTER_VALIDATE_IP	Validates value as IP address, optionally only IPv4 or IPv6 or not from private or reserved ranges.
FILTER_VALIDATE_MAC	Validates value as MAC address.
FILTER_VALIDATE_REGEXP	Validates value against regexp, a Perl-compatible regular expression.
FILTER_VALIDATE_URL	Validates value as URL

Example: We validate an integer using the **filter_var()** function:

```
<?php
$int=1234;

if(!filter_var($int,FILTER_VALIDATE_INT))
{
    echo "Integer is not valid";
}
else
{
    echo "Integer is valid";
}
```

Regular expression:

A regular expression is a sequence of characters that forms a search pattern. A regular expression can be a single character, or a more complicated pattern.

Unit 2: Advanced PHP and File Management

Regular Expression Functions

Function	Description	
preg_match()	Returns 1 if the pattern was found in the string and 0 if not	<pre>\$str = "I LIKE apple."; \$pattern = "/like/i"; echo preg_match(\$pattern, \$str); // Outputs 1</pre>
preg_match_all()	Returns the number of times the pattern was found in the string, which may also be 0	<pre>\$str = "It is an apple.I like apple"; \$pattern = "/apple/"; echo preg_match_all(\$pattern, \$str); // Outputs 2</pre>
preg_replace()	Returns a new string where matched patterns have been replaced with another string	<pre>\$str = " It is an apple.I like apple "; \$pattern = "/apple/i"; echo preg_replace(\$pattern, "orange", \$str); // Outputs " It is an orange.I like orange "</pre>

Regular Expression Patterns

Expression	Description
[abc]	Find one character from the options between the brackets
[^abc]	Find any character NOT between the brackets
[0-9]	Find one character from the range 0 to 9

Unit 2: Advanced PHP and File Management

Metacharacters

Metacharacter	Description
	Find a match for any one of the patterns separated by as in: cat dog fish
.	Find just one instance of any character
^	Finds a match as the beginning of a string as in: ^Hello
\$	Finds a match at the end of the string as in: World\$
\d	Find a digit
\s	Find a whitespace character
\b	Find a match at the beginning of a word like this: \bWORD, or at the end of a word like this: WORD\b
\uxxxx	Find the Unicode character specified by the hexadecimal number xxxx

Quantifiers

Quantifier	Description
{quantities}	
n+	Matches any string that contains at least one <i>n</i>
n*	Matches any string that contains zero or more occurrences of <i>n</i>

Unit 2: Advanced PHP and File Management

<code>n?</code>	Matches any string that contains zero or one occurrences of <i>n</i>
<code>n{x}</code>	Matches any string that contains a sequence of <i>X n</i> 's
<code>n{x,y}</code>	Matches any string that contains a sequence of <i>X</i> to <i>Y n</i> 's
<code>n{x,}</code>	Matches any string that contains a sequence of at least <i>X n</i> 's

Empty String

Checks that the field is not empty. If the user leaves the required field empty, it will show an error message.

```
if (empty ($_POST["name"])) {  
    $errMsg = "Error! You didn't enter the Name.";  
    echo $errMsg;  
} else {  
    $name = $_POST["name"];  
}
```

Validate String

checks that the field will contain only alphabets and whitespace, for example - name. If the name field does not receive valid input from the user

```
$name = $_POST ["Name"];  
if (!preg_match ("/^[a-zA-Z]*$/", $name) )  
{  
    $errMsg = "Only alphabets and whitespace are allowed.";  
    echo $errMsg;  
}  
else  
{  
    echo $name;  
}
```

Explanation:

❓ `"/^[a-zA-Z]*$/"`:

- This is the regular expression pattern used for the match.
- `^`: Anchors the match to the beginning of the string.

Unit 2: Advanced PHP and File Management

- **[a-zA-Z]**: Defines a character class that matches any uppercase (A-Z) or lowercase (a-z) letter.
- *****: Indicates that the preceding character class can occur zero or more times.
- **\$**: Anchors the match to the end of the string.
- Thus, this pattern will match a string that consists entirely of letters (both uppercase and lowercase) and can be of any length, including an empty string.

Validate Number

The field will only contain a numeric value.

For example - Mobile no

```
$mobilenos = $_POST ["Mobile_no"];  
if (!preg_match ("/^[0-9]*$/", $mobilenos) )  
{  
    $ErrMsg = "Only numeric value is allowed.";  
    echo $ErrMsg;  
} else  
{  
    echo $mobilenos;  
}
```

Explanation:

"/^[0-9]*\$/":

- This is the regular expression pattern used in the match.
- **^**: Anchors the pattern to the start of the string.
- **[0-9]**: This defines a character class that matches any digit from 0 to 9.
- *****: This quantifier indicates that the preceding element (digits) can occur zero or more times.
- **\$**: Anchors the pattern to the end of the string.
- So, this pattern matches a string that consists entirely of digits (or is empty).

Validate Email

A valid email must contain @ and . symbols

```
$email = $_POST ["Email"];  
$pattern = "/^[a-z0-9._-]+@[a-z0-9.-]+\.[a-z]{2,}$/i";  
if (!preg_match ($pattern, $email) )  
{  
    $ErrMsg = "Email is not valid.";
```

Unit 2: Advanced PHP and File Management

```
        echo $ErrMsg;
    }
    else
    {
        echo "Your valid email address is: " . $email;
    }
}
```

Explanation:

/:

- Delimiters that define the start and end of the pattern.

^[a-z0-9._-]+:

- Matches the beginning of the string with one or more characters that are either lowercase letters (a-z), digits (0-9), periods (.), underscores (_), or hyphens (-).

@:

- Matches the literal @ character.

[a-z0-9.-]+:

- Matches the domain part of the email with one or more lowercase letters (a-z), digits (0-9), periods (.), or hyphens (-).

\.[a-z]{2,}:

- Matches a period (.) followed by at least two lowercase letters (a-z). This allows for more than just the common 2-3 letter TLDs, accommodating newer or longer TLDs.

\$:

- Ensures that the pattern must end at this point.

i:

- The case-insensitive flag, allowing uppercase letters in the email.

Input Length Validation

The input length validation restricts the user to provide the value between the specified range, for Example - Mobile Number. A valid mobile number must have 10 digits.

```
$mobilenos = strlen($_POST["Mobile"]);
$length = strlen($mobilenos);
```

```
if ( $length != 10)
```


Unit 2: Advanced PHP and File Management

```
{
    $ErrMsg = "Mobile must have 10 digits.";
    echo $ErrMsg;
}
else
{
    echo "Your Mobile number is: " . $mobilenos;
}
```

Button Click Validate

Code validates that the user click on submit button and send the form data to the server one of the following method - get or post.

```
if (isset($_POST['submit']))
{
    echo "Submit button is clicked.";
    if ($_SERVER["REQUEST_METHOD"] == "POST")
    {
        echo "Data is sent using POST method ";
    }
}
else
{
    echo "Data is not submitted";
}
```

Unit 2: Advanced PHP and File Management

2.2.4 Parsing and Generating JSON with `json_encode()` and What is JSON

JSON stands for **Java Script Object Notation**. JSON is a standard lightweight data-interchange format which is quick and easy to parse and generate.

JSON, like XML, is a text-based format that's easy to write and easy to understand for both humans and computers, but unlike XML, JSON data structures occupy less bandwidth than their XML versions. JSON is based on two basic structures:

Object: This is defined as a collection of key/value pairs (i.e. `key:value`). Each object begins with a left curly bracket `{` and ends with a right curly bracket `}`. Multiple key/value pairs are separated by a comma `,`.

Array: This is defined as an ordered list of values. An array begins with a left bracket `[` and ends with a right bracket `]`. Values are separated by a comma `,`.

In JSON, keys are always strings, while the value can be a **string**, **number**, **true** or **false**, **null** or even an **object** or an **array**. Strings must be enclosed in double quotes `"` and can contain escape characters such as `\n`, `\t` and `\`.

JSON object:

```
{
  "book": {
    "name": "Harry Potter and the Goblet of Fire",
    "author": "J. K. Rowling",
    "year": 2000,
    "genre": "Fantasy Fiction",
    "bestseller": true
  }
}
```

JSON array:

```
{
  "fruits": [
    "Apple",
    "Banana",
    "Strawberry",
    "Mango"
  ]
}
```

Parsing JSON with PHP

JSON data structures are very similar to PHP arrays. PHP has built-in functions to encode and decode JSON data. These functions are `json_encode()` and `json_decode()`, respectively. Both functions only

Unit 2: Advanced PHP and File Management

works with UTF-8(Unicode Transformation Format - 8 bits) encoded (this code use for electronic communication in XML/Json/HTML5) string data.

Encoding JSON Data in PHP

In PHP the `json_encode()` function is used to encode a value to JSON format. The value being encoded can be any PHP data type except a resource, like a database or file handle.

Example to encode a PHP associative array into a JSON object:

```
<?php
// An associative array
$marks = array("Peter"=>65, "Harry"=>80, "John"=>78, "Clark"=>90);

echo json_encode($marks);
?>
```

Output

```
{"Peter":65,"Harry":80,"John":78,"Clark":90}
```

Similarly, you can encode the PHP indexed array into a JSON array, like this:

Example

```
<?php
// An indexed array
$colors = array("Red", "Green", "Blue", "Orange", "Yellow");

echo json_encode($colors);
?>
```

Output:

```
["Red","Green","Blue","Orange","Yellow"]
```

Note: force `json_encode()` function to return an PHP indexed array as JSON object by using the `JSON_FORCE_OBJECT` option, as shown in the example below:

```
<?php
// An indexed array
$colors = array("Red", "Green", "Blue", "Orange");

echo json_encode($colors, JSON_FORCE_OBJECT);
?>
```

Unit 2: Advanced PHP and File Management

The output of the above example will look like this:

```
{"0":"Red","1":"Green","2":"Blue","3":"Orange"}
```

As you can see in the above examples a non-associative array can be encoded as array or object. However, an associative array always encoded as object.

Note: error may be occurs in JSON_FORCE_OBJECT due lower version of php.

Decoding JSON Data in PHP

Decoding JSON data is as simple as encoding it. You can use the PHP `json_decode()` function to convert the JSON encoded string into appropriate PHP data type.

Example to decode or convert a JSON object to PHP object.

```
<?php
// Store JSON data in a PHP variable
$json = '{"Peter":65,"Harry":80,"John":78,"Clark":90}';

var_dump(json_decode($json));
?>
```

Output:

```
object(stdClass)#1 (4) { ["Peter"]=> int(65) ["Harry"]=> int(80) ["John"]=> int(78) ["Clark"]=> int(90) }
```

stdClass: A generic class for creating objects without a predefined structure.

#1: An internal identifier for this instance of stdClass.

(4): Indicates the number of properties in the object.

By default the `json_decode()` function returns an object. However, you can optionally specify a second parameter **\$assoc** which accepts a boolean value that when set as `true` JSON objects are decoded into associative arrays. It is `false` by default.

Example to decode or convert a JSON object to PHP array.

```
<?php
// Store JSON data in a PHP variable
$json = '{"Peter":65,"Harry":80,"John":78,"Clark":90}';
var_dump(json_decode($json, true));
?>
```

Unit 2: Advanced PHP and File Management

Output

```
array(4) { ["Peter"]=> int(65) ["Harry"]=> int(80) ["John"]=> int(78) ["Clark"]=> int(90) }
```

Now let's check out an example that will show you how to decode the JSON data and access individual elements of the JSON object or array in PHP.

```
<?php
// Assign JSON encoded string to a PHP variable
$json = '{"Peter":65,"Harry":80,"John":78,"Clark":90}';

// Decode JSON data to PHP associative array
$arr = json_decode($json, true);
// Access values from the associative array
echo $arr["Peter"]; // Output: 65
echo $arr["Harry"]; // Output: 80
echo $arr["John"]; // Output: 78
echo $arr["Clark"]; // Output: 90

// Decode JSON data to PHP object
$obj = json_decode($json);
// Access values from the returned object
echo $obj->Peter; // Output: 65
echo $obj->Harry; // Output: 80
echo $obj->John; // Output: 78
echo $obj->Clark; // Output: 90
?>
```

2.3 Cookies, Sessions, and Emails

2.3.1 Creating and accessing cookies using setcookie() and \$_COOKIE

A cookie is a small text file that lets you store a small amount of data (nearly 4KB) on the user's computer. They are typically used to keeping track of information such as username that the site can retrieve to personalize the page when user visit the website next time.

Set cookies: Use to set cookies

`setcookie(name, value, expire, path, domain, secure);`

Here,

name	The name of the cookie.
value	The value of the cookie. Do not store sensitive information
expires	The expiry date. After this time cookie will become inaccessible. The default value is 0.
path	Specify the path on the server for which the cookie will be available. If set to /, the cookie will be available within the entire domain.
domain	Specify the domain for which the cookie is available to e.g www.example.com.
secure	This field, if present, indicates that the cookie should be sent only if a secure HTTPS connection exists.

Example: create a cookie named `username` and assign the value `John Carter` to it. It also specify that the cookie will expire after 30 days (30 days * 24 hours * 60 min * 60 sec).

```
setcookie("username", "John Carter", time()+30*24*60*60);
```

Access cookies:

`$_COOKIE` super global variable is used to retrieve a cookie value.

Example:

```
echo $_COOKIE["username"];
```

Removing Cookies:

Delete a cookie by calling the same `setcookie()` function with the cookie name and any value or set the expiration date in the past.

Unit 2: Advanced PHP and File Management

Example:

```
setcookie("username", "", time()-3600);
```

Example:

```
<?php
$cookie_name = "user";
$cookie_value = "John Doe";
setcookie($cookie_name, $cookie_value, time() + (86400 * 30), "/"); // 86400 = 1 day
?>

<html>
<body>

<?php
if(!isset($_COOKIE[$cookie_name]))
{
    echo "Cookie named '" . $cookie_name . "' is not set!";
}

else
{
    echo "Cookie '" . $cookie_name . "' is set!<br>";
    echo "Value is: " . $_COOKIE[$cookie_name];
}
?>

</body>
</html>
```

2.3.2 Session management with session_start() and \$_SESSION

session stores data on the server rather than user's computer. In a session based environment, every user is identified through a unique number called session identifier or SID. Default session expired after 24 minutes which can be modify by changing in php.ini file.

Which resolve two problems in cookies:

- 1) Store data using cookies but it has some security issues. Since cookies are stored on user's computer it is possible for an attacker to easily modify a cookie content to insert potentially harmful data in your application that might break your application.

Unit 2: Advanced PHP and File Management

- 2) Also every time the browser requests a URL to the server, all the cookie data for a website is automatically sent to the server within the request. It means if you have stored 5 cookies on user's system, each having 4KB in size, the browser needs to upload 20KB of data each time the user views a page, which can affect your site's performance.

Start session:

store any information in session variables, you must first start up the session.

Example:

```
session_start();
```

Storing Session Data

store all your session data as key-value pairs in the `$_SESSION[]` superglobal array. The stored data can be accessed during lifetime of a session.

Syntax:

```
$_SESSION["sessionname"] = value;
```

Example:

```
$_SESSION["firstname"] = "Peter";
```

Accessing Session Data

To access the session data we set on our previous example from any other page on the same web domain.

```
echo 'Hi, ' . $_SESSION["firstname"];
```

Destroying a Session

If you want to remove certain session data

```
unset($_SESSION["session name"]);
```

Example:

```
unset($_SESSION["firstname"])
```


Unit 2: Advanced PHP and File Management

Note:

To destroy a session completely, simply call the `session_destroy()` function.

```
<?php
session_start();
?>
<!DOCTYPE html>
<html>
<body>

<?php
// Echo session variables that were set on previous page
echo "Favorite color is " . $_SESSION["favcolor"] . "<br>";
echo "Favorite animal is " . $_SESSION["fanimal"] . ".";
?>

</body>
</html>
```

2.3.3 Sending emails using the mail() function

Using `mail()` function for creating and sending email messages to one or more recipients

Syntax:

`mail(to, subject, message, headers, parameters)`

To	The recipient's email address.
Subject	Subject of the email to be sent. This parameter i.e. the subject line cannot contain any newline character (\n).
Message	Defines the message to be sent. Each line should be separated with a line feed-LF (\n). Lines should not exceed 70 characters.
Headers	This is typically used to add extra headers such as "From", "Cc", "Bcc". The additional headers should be separated with a carriage return plus a line feed-CRLF (\r\n).

Unit 2: Advanced PHP and File Management

(optional)	
Parameters (optional)	Used to pass additional parameters.

Example:

```
<?php $to = 'maryjane@email.com';  
  
$subject = 'Marriage Proposal';  
  
$message = 'Hi Jane, will you marry me?';  
  
$from = 'peterparker@email.com'; // Sending email  
  
if(mail($to, $subject, $message))  
{ echo 'Your mail has been sent successfully.'; }  
  
Else  
  
{ echo 'Unable to send email. Please try again.'; }  
  
?>
```

Unit 2: Advanced PHP and File Management

2.4 OOP and Exception Handling in PHP

2.4.1 Classes and objects in Php

Classes and objects are the two main aspects of object-oriented programming.

Class : Fruit

Objects: Apple,Banana,Mango

a class is a template for objects, and an object is an instance of a class.

Define a Class

A class is defined by using the class keyword, followed by the name of the class and a pair of curly braces ({}).

Syntax

```
<?php
class classname
{
Variables/properties declaration
Methods
    // code goes here...
}
?>
```

Example

```
<?php
class Fruit
{
    // Properties
    public $name;
    public $color;

    // Methods
    function set_name($name)
```

Unit 2: Advanced PHP and File Management

```
{
    this->name = $name;
}
function get_name()
{
    return $this->name;
}
}
?>
```

Define Objects

Classes are nothing without objects! We can create multiple objects from a class. Each object has all the properties and methods defined in the class, but they will have different property values.

Objects of a class is created using the new keyword.

Syntax:

```
objectname = new classname;
```

Example:

```
<?php
class Fruit {
    // Properties
    public $name;
    public $color;

    // Methods
    function set_name($name) {
        $this->name = $name;
    }
    function get_name() {
        return $this->name;
    }
}

$a = new Fruit();
$b = new Fruit();
$a->set_name('Apple');
$b->set_name('Banana');
```

Unit 2: Advanced PHP and File Management

```
echo $a->get_name();  
echo "<br>";  
echo $b->get_name();  
?>
```

2.4.2 Using Constructors and Property Visibility

- A **constructor** is a special function called automatically when an object is created. In PHP, it is named `__construct()`.
- **Visibility** controls access to class properties and methods:
 - `public`: accessible from anywhere.
 - `private`: accessible only inside the class.
 - `protected`: accessible within the class and its child classes.

```
<?php  
class Car {  
    private $model;  
  
    // Constructor  
    public function __construct($model) {  
        $this->model = $model;  
    }  
  
    public function showModel() {  
        echo "Car model is: " . $this->model;  
    }  
}  
  
$car1 = new Car("Toyota");  
$car1->showModel();  
?>
```

2.4.3 Inheritance and Method Overriding

- **Inheritance** allows a class (child) to inherit properties and methods from another class (parent).
- **Method Overriding** means redefining a parent class method in the child class.

Syntax:

```
class ChildClass extends ParentClass
```

Unit 2: Advanced PHP and File Management

Example:

```
<?php
class Animal {
    public function makeSound() {
        echo "Animal sound";
    }
}

class Dog extends Animal {
    // Overriding method
    public function makeSound() {
        echo "Bark Bark!";
    }
}

$pet = new Dog();
$pet->makeSound(); // Output: Bark Bark!
?>
```

2.4.4 Exception handling: try, catch, finally, throw

What is an Exception?

An exception is an object that describes an error or unexpected behaviour of a PHP script.

Exceptions are thrown by many PHP functions and classes.

The try...catch... Statement

The try...catch statement to catch exceptions and continue the process..

Syntax

```
try
{
    code that can throw exceptions
}

catch(Exception $e)
```

Unit 2: Advanced PHP and File Management

```
{  
    code that runs when an exception is caught  
}
```

User defined functions and classes can also throw exceptions.

Throwing an Exception

The throw statement allows a user defined function or method to throw an exception. When an exception is thrown, the code following it will not be executed.

If an exception is not caught, a fatal error will occur with an "Uncaught Exception" message.

Syntax:

```
throw new Exception("msg");
```

Example:

```
<?php  
function division($x,$y)  
{ if ($y==0)  
    throw new exception("Answer is invalid. Division by zero");  
    else  
    return $x/$y;  
}  
try  
{  
    echo division(10,2);  
    echo division(5,0);  
}  
catch(Exception $e)  
{  
    echo $e->getMessage();  
}  
?>
```